
① Basic:-

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *next;
};
```

```
void linkListTraversal(struct node *ptr)
{
    while (ptr != NULL) {
        printf("\ndata: %d", ptr->data);
        ptr = ptr->next;
    }
}
```

```
int main() {  
    struct node *head;  
    struct node *second;  
    struct node *third;
```

```
head = (struct node *) malloc(sizeof
(struct node));
```

Second = " " _ _ _ _

third = " " " " " "

head $\rightarrow$ data = 1;

head $\rightarrow$ next = Second;

second $\rightarrow$ data = 2;

second $\rightarrow$ next = third;

Kind $\rightarrow$ data = 3;

third $\rightarrow$ next = NULL;

printf("Link list");

Link List Traversal (head);

```
return 0;
```

3

$$O/P \Rightarrow$$

Linked list

data: 1

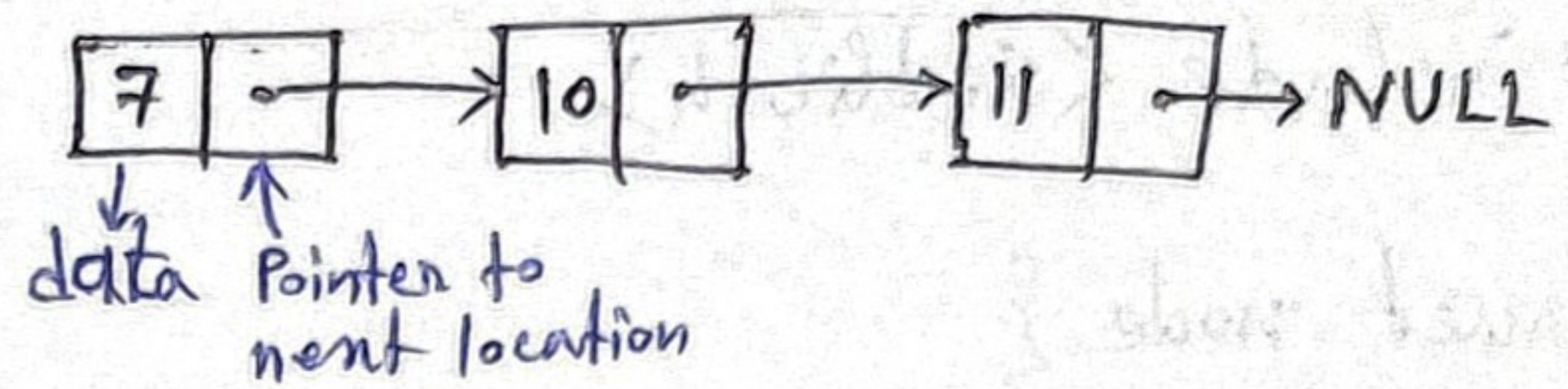
data : 2

data: 3

Link lists are similar to array or linear data structure

7	10	11	12	18	22
---	----	----	----	----	----

↑ in arrays elements are stored in contiguous memory locations.



In linked lists, elements are stored in non contiguous memory locations.

Why Linked Lists :-

Memory and the capacity of an array remain fixed, while in linked lists, we can keep adding and removing elements without any capacity constraint.

Drawbacks :-

- * Extra Memory space for pointers is required for every node.

- * Random access is not allowed as elements are not stored in contiguous memory locations.

Implementation :-

```
struct node{
```

```
int data;
```

```
struct Node *next;
```

3;

Self referencing structure

data: 1

data : 2

data: 3

* Insertion in a Linked List :-

Case 1 :- Insert at the beginning

Case 2 :- Insert in between

Case 3 :- Insert at the end

Case 4 :- Insert after a node

Case 1 :-

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct node {
```

```
    int data;
```

```
    struct node *next;
```

```
};
```

```
struct node *insertAtFirst(struct node *head, int data) {
```

```
    struct node *ptr = (struct node *) malloc(sizeof(struct node));
```

```
    ptr->next = head;
```

```
    ptr->data = data;
```

```
    return ptr;
```

```
}
```

```
void linklistTraversal(struct node *ptr) {
```

```
    while (ptr != NULL) {
```

```
        printf("\ndata: %d", ptr->data);
```

```
        ptr = ptr->next;
```

```
    }
```

```
}
```

```
int main() {
```

```
    struct node *head;
```

```
    struct node *second;
```

```
    struct node *third;
```

```
    head = (struct node *) malloc(sizeof(struct node));
```

```
    second = (struct node *) malloc(sizeof(struct node));
```

```
    third = (struct node *) malloc(sizeof(struct node));
```

head → data = 1;
head → next = second

second → data = 2;
second → next = third;
third → data = 3;
third → next = NULL;

printf("Linked list before insertion");
linkListTraversal(head);

head = insertAtFirst(head, 56);

printf("\n Linked list after insertion");
linkListTraversal(head);

return 0;

}

Case 2:-

// All are same except insert function & function call

struct node *insertAtIndex(struct node *head, int data, int index){

struct node *ptr = (struct node *) malloc (sizeof (struct node));

struct node *p = head;

int i = 0;

while (i != (index - 1)) {

p = p → next;

i++;

}

ptr → data = data;

ptr → next = p → next;

p → next = ptr;

return head;

}

void linkListTraversal (struct node *ptr) { } // same

int main () {

// all same

head = insertAtIndex (head, 56, 2);

}

O/P →

Linked list before insertion

data: 1

data: 2

data: 3

Linked list after insertion

data: 56

data: 1

data: 2

data: 3

O/P →

Linked list before insertion

data: 1

data: 2

data: 3

Linked list after insertion

data: 1

data: 2

data: 56

data: 3

Case 3:-

// All same

```
struct node *insertAtLast (struct node *head, int data) {  
    struct node *ptr = (struct node *) malloc (sizeof (struct node));  
    struct node *p = head;  
    while (p->next != NULL)  
        p = p->next;  
  
    ptr->data = data;  
    ptr->next = NULL;  
    p->next = ptr;  
    return head;  
}
```

// All same

```
int main () {
```

// All same

```
    head = insertAtLast (head, 56);  
}
```

Case 4:-

// All same

```
struct node *insertAfterNode (struct node *node, int data) {  
    struct node *ptr = (struct node *) malloc (sizeof (struct node));  
    ptr->data = data;  
    ptr->next = node->next;  
    node->next = ptr;  
}
```

// All same

```
int main () {
```

// All same

```
    *insertAfterNode (second, 56);
```

```
}
```

o/p ⇒ Linked list before insertion

data : 1

data : 2

data : 3

Linked list after insertion

data : 1

data : 2

data : 3

data : 56

o/p ⇒

Linked list before insertion

data : 1

data : 2

data : 3

Linked list after insertion

data : 1

data : 2

data : 56

data : 3

* Deletion of a linked list

case 1: Deleting the first node

case 2: Deleting ~~the~~ a node in between

case 3: Delete the last node

case 4: Delete a node with a given value/key

case 1:-

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct node {
```

```
    int data;
```

```
    struct node *next;
```

```
};
```

```
struct node *deleteFirst(struct node *head) {
```

```
    struct node *p = (struct node *) malloc(sizeof(struct node));
```

```
    p = head;
```

```
    head = head->next;
```

```
    free(p);
```

```
    return head;
```

```
}
```

```
void linkListTraversal(struct node *ptr) {
```

```
    while (ptr != NULL) {
```

```
        printf("\n data: %d", ptr->data);
```

```
        ptr = ptr->next;
```

```
    }
```

```
}
```

```
int main() {
```

```
    struct node *head;
```

```
    struct node *second;
```

```
    struct node *third;
```

```
    struct node *fourth;
```

```

head = (struct node *) malloc(sizeof(struct node));
second = (struct node *) malloc(sizeof(struct node));
third = (struct node *) malloc(sizeof(struct node));
fourth = (struct node *) malloc(sizeof(struct node));

```

```

head → data = 1;
head → next = second;
second → data = 2;
second → next = third;
third → data = 3;
third → next = fourth;
fourth → data = 4;
fourth → next = NULL;

```

o/p →
Linked list before deletion

data: 1

data: 2

data: 3

data: 4

Linked list after deletion

data: 2

data: 3

data: 4

```
printf("Linked list before Deletion");
```

```
linkListTraversal(head);
```

```
head = deleteFirst(head);
```

```
printf("\n\nLinked list after deletion");
```

```
linkListTraversal(head);
```

```
return 0;
```

```
}
```

Case 2:-

// All same

```
struct node *deleteIndex(struct node *head, int index) {
```

```
    struct node *p = (struct node *) malloc(sizeof(struct node));
```

```
    struct node *q = (struct node *) malloc(sizeof(struct node));
```

```
    p = head;
```

```
    int i = 0;
```

```
    while (i != index - 1) {
```

```
        p = p → next;
```

```
        i++;
```

```
}
```

```

q = p → next;
p → next = q → next;
free(q);
return head;
}

```

// All same

```

int main() {

```

// All same

```

head = deleteIndex(head, 2);

```

```

}

```

Case 3:-

// All same

```

struct node *deleteLast(struct node *head) {

```

```

    struct node *p = (struct node *) malloc(sizeof(struct node));

```

```

    struct node *q = (struct node *) malloc(sizeof(struct node));

```

```

    q = head;

```

```

    p = q → next;

```

```

    while (p → next != NULL) {

```

```

        p = p → next;

```

```

        q = q → next;

```

```

    }

```

```

    q → next = NULL;

```

```

    free(p);

```

```

    return head;

```

```

}

```

// All same

```

int main() {

```

// All same

```

head = deleteLast(head);

```

```

}

```

O/P ⇒

Linked list before deletion

data: 1 → 2 → 3 → 4

data: 2 → 3 → 4

data: 3 → 4

data: 4

Linked list after deletion

data: 1

data: 2

data: 4

O/P ⇒

Linked list before deletion

data: 1

data: 2

data: 3

data: 4

Linked list after deletion

data: 1

data: 2

data: 3

Case 4:-

// All same

```
struct node *deleteNode(struct node *head, struct node *second);  
struct node *p = (struct node *) malloc(sizeof(struct node));  
struct node *q = (struct node *) malloc(sizeof(struct node));
```

q = head;

p = q → next;

while (p != second) {

 p = p → next;

 q = q → next;

}

q → next = p → next;

free(p);

return head;

}

// All same

```
int main () {
```

// All same

head = deleteNode(head, second);

}

o/p ⇒

Linked list before deletion

data : 1

data : 2

data : 3

data : 4

Linked list after deletion

data : 1

data : 3

data : 4